

Lab06 - Threaded Matrix Multiplier

Général

- Auteurs: Fabien Léger & Aymeric Bonny
- Cours: PCO, HEIG-VD
- Date: 19.12.2025

Problématique

Faire un programme qui résout la multiplication matricielle entre une matrice A et B et mettre le résultat dans une matrice C. Le programme utilisera un buffer Producteur-Consommateur utilisant un moniteur de Hoare sous la forme de `PcoHoareMonitor` duquel héritera le buffer.

Implémentation

Buffer

Nous avons décidé de commencer par le buffer, car celui-ci semblait la base.

Le buffer a une fonction `sendJob()` pour envoyer des jobs dans le buffer et une fonction `getJob()` pour les récupérer. On pourra ainsi envoyer des jobs depuis notre thread principal pour que nos threads travailleurs les reprennent par la suite.

Une `std::queue` sous le nom de `buffer` a été choisi comme buffer pour faciliter l'utilisation et garder un ordre FIFO. Bien que cela ne soit pas utile dans notre implémentation, si l'on voulait avoir plusieurs threads accédant à notre buffer la queue FIFO aiderait grandement et avons donc décider de partir là-dessus.

La fonction `requestStop()` permet d'arrêter le buffer et de libérer tous les threads bloqués. Elle sert surtout lors de la destruction du buffer.

La fonction `waitForFinishedJobs()` permet de bloquer notre thread principal jusqu'à ce que tous les jobs soient finis

La fonction `addJobAndCheckIfFinished()` permet d'incrémenter notre nombre de jobs finis de manière concurrent tout en regardant si on a atteint notre but. Dans le cas où c'est vrai, on fera un signal au thread principal.

Les fonctions `acquireBuffer()` et `releaseBuffer()` servent à n'avoir qu'un seul `multiply()` utilisant le buffer. Cela permet d'avoir une fonction `multiply()` ré-entrante. Lors de multiples appels, seul le premier sera laissé passé tandis que les autres appels seront mis en attente dans `acquireBuffer()` jusqu'à ce que ce premier thread ait fini et qu'il appelle `releaseBuffer()`.

Voici également un tableau récapitulatif des variables condition :

Nom	Utilité
waitSendJob	Fait attendre les fonctions voulant envoyer un job au buffer
waitGetJob	Fait attendre les fonctions voulant recevoir un job du buffer
finishedJobs	Fait attendre le <code>multiply()</code> avec la propriété jusqu'à ce que tous les jobs soient finis
ownership	Fait attendre les autres <code>multiply()</code> , car il y en a déjà un qui le possède

Multiply

La fonction n'est pas très longue. Il suffit de checker les différentes valeurs pour être sûr que cela a du sens. Ensuite, on crée les différents "sous blocs" de la matrice C.

Nous avons ajouté à `ComputeParameters` les paramètres suivants :

- row: l'index de la ligne du sous-bloc auquel commencer
- col: l'index de la colonne du sous-bloc auquel commencer
- blockSize: la taille du bloc à calculer (et des tous les sous-blocs)

MultiplySimple

La fonction est lancée dès la création du `ThreadedMatrixMultiplier`. Dans le constructeur, nous créons nos threads qui utilisent `multiplySimple()` comme point de départ.

C'est un travailleur qui attend que soit-on lui dit de se stopper soit que le buffer se stoppe. Une fois que le travailleur a un job, il tournera sur les cases résultats de C pour y écrire ses calculs entre les matrices A et B. Une fois le travail finit, le travailleur incrémentera le compteur de jobs finis via la fonction `addJobAndCheckIfFinished()` et comme dit avant regardera si tous les jobs ont été fait.

Destructeurs

Le destructeur de `Buffer` fait bien attention à ce que tous les threads soient relâchés via la méthode `requestStop()` et que les threads qui entrent par la suite soient renvoyés.

`~ThreadedMatrixMultiplier()` appelle également cette méthode afin de libérer les threads. Puis, il demande aux threads de s'arrêter en leur envoyant un `thread->requestStop()`. Finalement, le destructeur attend que tous les threads aient join avant de se détruire définitivement.

Tests

Voici la liste des tests qui ont été ajoutés :

Suite	Test	Objectif
MultiplierStudent	MoreThreadsThanBlocks	Test avec plus de threads que de blocs
MultiplierStudent	SingleBlockPerRow	Test avec 1 bloc par ligne
MultiplierStudent	AsManyBlocksAsMatrixSize	Test avec un bloc pour une case
ZeroValues	MatrixSizeZero	Test avec une matrice vide

Il aurait également été bien de tester des valeurs négatives et nulles pour le nombre de threads et le nombre de blocs par lignes, mais cela ne semblait pas possible de premier abord. Nous sommes donc restés sur ces tests quelque peu simples tout en testant des limites intéressantes.

Conclusion

Nous avons eu beaucoup de problèmes pendant ce laboratoire. La donnée était dure à comprendre et certains aspects comme des informations redondantes avec `nbBlocksPerRow` apparaissant et dans le constructeur et dans `multiply` ont rendu la tâche difficile.

Nous croyions qu'il fallait créer des sous-blocs de la matrice résultat C. Nous étions donc parti sur une solution bien plus compliquée que prévu. Après avoir tout codé et cela ne marchant pas, nous nous sommes rendu compte du problème. Il nous a fallu plusieurs jours entiers sur le labo afin de rendre le code fonctionnel.

Toutefois, ce fut un bon entraînement au niveau du moniteur de Hoare. Ceux-ci permettent de faciliter grandement la concurrence et de "éviter de trop penser" en ne faisant qu'entrer dans des moniteurs et en vérifiant des variables. Il est donc plus facile de coder et d'éviter des erreurs.

Un plus aurait été de poser plus de questions d'implémentation, mais il est aussi à nous de comprendre par nous même et la balance est donc compliquée.